

Software Final Question Solve: Fall2020

1. a. The size of the code for the project in KLOC (kilo lines of code) is last four digit of your registration number. Project manager hires developers of very low quality but a lot of experience in programming language. Calculate the effort (E) and scheduled time for development (D) according to cost drivers table for semi-detached and embedded type project.
[If your registration number is 18101001, then KLOC will be 1001]

Table 1: Cost drivers

Cost Drivers	Very Low	Low	Nominal	High	Very High	Extra High
AEXP	1.29	1.13	1.00	0.95	0.82	0.78
LEXP	2.29	2.13	1.00	0.95		

$ID = 18101094$
 $\therefore KLOC = 1094$
 Here, $EAF = 1.29 \times 0.95$
 $= 1.225$
 For semi detached project,
 $a = 3.0$
 $b = 1.12$
 $c = 2.5$
 $d = 0.35$

$$\text{Effort (E)} = a (KLOC)^b \times EAF$$

$$= 3.0 (1094)^{1.12} \times 1.225$$

$$= 9310 \text{ person-month}$$

$$\text{Development time (D)} = c(E)^d$$

$$= 2.5 (9310)^{0.35}$$

$$= 61.21 \text{ month}$$

Now, For embedded type project,
 $a = 2.8$
 $b = 1.20$
 $c = 2.5$
 $d = 0.32$

$$\therefore \text{Effort (E)} = a (KLOC)^b \times EAF$$

$$= 2.8 (1094)^{1.20} \times 1.225$$

$$= 12415.91 \times 1.225$$

$$= 15209 \text{ person-month}$$

$$\text{Development time (D)} = c(E)^d$$

$$= 2.5 \times (15209)^{0.32}$$

$$= 54.47 \text{ month}$$

Spring2024

1. ✓ a. Assume that you are accessing an online banking portal over public Wi-Fi at a coffee shop. Suddenly, you suspect that someone might be intercepting your communication. Describe the methods by which a Man-In-The-Middle (MITM) attack can be prevented in this situation. [5] [CO1]
- ✓ b. Differentiate Microservices Architecture and Monolithic Architecture in the context of software engineering. [5] [CO1]

Or

- a. You are developing a web application that allows users to submit comments on blog posts. A user has reported that malicious scripts are being executed when they view specific comments. Describe the methods by which you can prevent Cross-Site Scripting (XSS) attacks in this situation. [5] [CO1]
 - b. Differentiate Service Oriented Architecture and Microservices Architecture in the context of software engineering. [5] [CO1]
2. ✓ a. Illustrate the interaction flow sequence of the MVC, MVP, and MVVM architectural patterns. Additionally, identify which of these patterns are best suited for different environments, specifically in the context of web applications, mobile applications, and applications requiring data binding. [5] [CO1]
 - ✓ b. Explain how the Client-Server Model operates for a web application, providing an appropriate diagram to illustrate the interaction between the client and the server hosting the web page. [5] [CO1]

Q1(a): Man-In-The-Middle (MITM) Attack Prevention

[Marks: 5 | CO1]

◆ Situation:

Accessing an online banking portal using public Wi-Fi may expose you to MITM attacks, where an attacker intercepts communication between client and server.

◆ Methods to Prevent MITM Attack:

Method	Explanation	Mark
1. Use HTTPS (SSL/TLS Encryption)	Ensure the website uses <code>https://</code> . TLS encrypts all communication between browser and bank server.	1

Method	Explanation	Mark
2. Avoid Public Wi-Fi or Use VPN	VPN encrypts all network traffic, even over public Wi-Fi, preventing sniffing.	1
3. Verify Digital Certificates	Check the site's SSL certificate for authenticity (e.g., valid padlock icon).	1
4. Use Two-Factor Authentication (2FA)	Even if credentials are intercepted, 2FA prevents unauthorized access.	1
5. Keep Device & Browser Updated	Security patches remove vulnerabilities that attackers can exploit.	

(a) Preventing Cross-Site Scripting (XSS) Attacks

[Marks: 5 | CO1]

Method	Explanation	Mark
1. Input Validation & Sanitization	Remove or escape <code><script></code> tags and special characters before storing input.	1
2. Output Encoding	Encode HTML entities before rendering user input in browser.	1
3. Content Security Policy (CSP)	Use HTTP headers to restrict where scripts can be loaded from.	1
4. HTTPOnly & Secure Cookies	Prevent JavaScript from accessing session cookies.	1
5. Framework-level Security Tools	Use frameworks' built-in escaping libraries (e.g., Django, Spring).	

Q2(a): MVC, MVP, and MVVM Patterns

[Marks: 5 | CO1]

◆ Flow Interaction and Environment Suitability

Pattern	Flow of Interaction	Best Suited For
MVC (Model–View–Controller)	Controller handles input → updates Model → View reflects changes.	Web Applications
MVP (Model–View–Presenter)	View interacts with Presenter → Presenter updates Model and View.	Mobile & Desktop Apps
MVVM (Model–View–ViewModel)	View binds directly to ViewModel via data binding.	Applications needing real-time UI updates (e.g., WPF, Angular, Flutter)

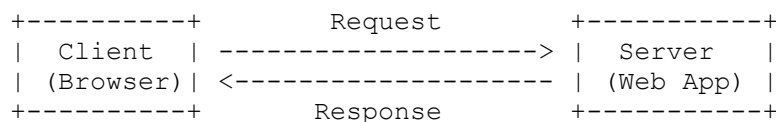
Q2(b): Client-Server Model (with Diagram)

[Marks: 5 | CO1]

◆ Concept:

A **client-server model** is a distributed application structure where the **client** sends a request to the **server**, and the **server** processes it and returns a response.

◆ Diagram:



◆ Explanation:

1. **Client:** Front-end interface (e.g., browser) that initiates communication. (1)
2. **Server:** Hosts data and application logic. (1)
3. **Interaction:** Client sends HTTP request → server processes → returns HTML/JSON response. (2)
4. **Example:** A user accessing a website like online banking. (1)

Spring 2021

The size of the code for the project in KLOC (kilo lines of code) is **last four digit** of your registration number. Project manager hires some developers. Their analytical capability is very high but experience in programming language is low. Surprisingly they have a very little experience on virtual machine. Calculate the effort (E) and scheduled time for development (D) according to cost drivers table for **embedded** type project.

[If your registration number is 18101001, then KLOC will be 1001]

Table 1: Cost drivers

Cost Drivers	Very Low	Low	Nominal	High	Very High
AEXP	3.29	3.13	3.00	2.50	1.52
LEXP	2.90	3.30	3.50	4.95	5.56
ACAP	3.50	4.50	5.35	6.15	7.37
VEXP	1.50	2.25	3.0	3.75	3.88

2. No. Qw Ans (a)

Here, my registration ID = 18201093

$$KLOC = 1093$$

Here, EAF = ACAP ^{high} ~~low~~ x LEXP ~~low~~ x VEXP ^{very} ~~low~~

$$= \frac{2.37}{2.37} \times 3.30 \times 1.50$$

$$= \frac{11.25}{11.25} \times 36.9815$$

For embedded type project,

$$a = \frac{2.8}{2.8} \times 2.5$$

$$b = 1.2$$

$$c = 2.5$$

$$d = 0.32$$

$$\text{So, Effort (E)} = a (KLOC)^b \times EAF$$

$$= 2.8 \times (1093)^{1.2} \times \frac{11.25}{11.25} \times 36.9815$$

$$= \frac{261166.1664}{261166.1664} \text{ person-month}$$

$$= 427732.1936 \text{ person-month. (Ans.)}$$

(P.T.O)

$$\text{Development time (D)} = c (E)^d$$

$$= 2.5 \times \left(\frac{427732.1936}{261166.1664} \right)^{0.32}$$

$$= \frac{185.817}{185.817} \text{ month}$$

$$= 158.458$$

(Ans.)

$$\text{Avg. resource size} = E/D$$

$$= \frac{261166.1664}{158.458} \text{ MM} = \frac{427732.1936}{158.458}$$

$$= \frac{1930.02 \text{ MM}}{1930.02 \text{ MM}} = 2699.34 \text{ MM}.$$

$$\text{Productivity of software} = \frac{KLOC}{E} \text{ KLOC/MM}$$

$$= \frac{1093}{261166.1664} \text{ KLOC/MM}$$

$$= \frac{1093}{427732.1936} \text{ KLOC/MM}$$

$$= 0.002438 \text{ KLOC/MM.}$$

Spring2020

A defect, which could have been removed during the initial stage, is removed in a later stage.

- I. How does this affect the cost? Justify every SDLC level of cost.
- II. What are the relationships with verification and validation process

I. Effect of Removing Defects in Later Stages

When a defect is detected **late** in the Software Development Life Cycle (SDLC), the **cost of fixing it increases exponentially**.

This happens because more components depend on that defect as development progresses, so more effort, time, and resources are needed to correct and retest.

◆ Cost Justification at Each SDLC Level

SDLC Phase	When Defect Introduced or Detected	Relative Cost (approx.)	Justification
1. Requirement Phase	If found here, cost is lowest (1x)	✓ 1x	Easy to change textual documents or diagrams. No code or design affected.
2. Design Phase	If not found earlier, cost increases	✓ 5x	Need to redesign architecture, interfaces, and update documentation.
3. Coding / Implementation	Defects now affect code logic	✓ 10x	Developers must rewrite and recompile code, retest modules.
4. Testing Phase	Found during system or integration testing	✓ 50x	Requires debugging, re-coding, re-testing, and documentation updates.
5. Deployment / Maintenance	Found after delivery	✓ 100x or more	Requires patch releases, downtime, customer dissatisfaction, and financial loss.

II. Relationship with Verification and Validation

◆ Verification

- **Definition:** The process of ensuring the product is being built **correctly** (i.e., conforms to specifications).
- **Objective:** Detect defects **early** during requirement and design stages.
- **Examples:** Reviews, walkthroughs, inspections, static analysis.

→ **Relation:** Early *verification* helps find defects before coding — thus reducing cost drastically.

◆ Validation

- **Definition:** The process of ensuring the **right product** is built — meets user needs and expectations.
- **Objective:** Detect defects **later** during testing and deployment.
- **Examples:** Functional testing, acceptance testing, usability testing.

→ **Relation:** *Validation* ensures correctness from a user's perspective, but if defects reach this stage, fixing them becomes very expensive.

Fall2024

QUESTION 1 [10 MARKS]

- a. Why is high cohesion and low coupling considered desirable? Provide an example to support your explanation. [5]
- b. "Tightly coupled classes are difficult to maintain, test, and reuse". Explain how dependency injection addresses this issue and improves software design. [5]

OR

- a. Explain the Model-View-Controller (MVC) architectural pattern in software design. Describe the roles of each component Model, View, and Controller and how they interact with one another. [5]
- b. What are the SOLID design principles in software engineering? Clearly explain what each letter in the acronym (S, O, L, I, D) stands for, and provide a brief explanation and example for any two of the principles. [5]

QUESTION 2 [10 MARKS]

- a. What are the objectives of the Verification and Validation (V&V) process, and why is it not possible to achieve full validation through inspections alone? [5]
- b. Describe three black-box testing techniques. For each, explain the principle behind it and what types of errors it helps to identify. [5]

? QUESTION 1 [10 Marks]

(a) Why is High Cohesion and Low Coupling Desirable?

[5 Marks | CO1]

◆ Explanation

Concept	Definition	Desirability
Cohesion	The degree to which the elements inside a module/class work together to perform a single, well-defined task.	High Cohesion means the module has a clear, single purpose → easier to understand, maintain, and reuse.
Coupling	The degree of dependency between different modules/classes.	Low Coupling means changes in one module have minimal effect on others → improves flexibility and scalability.

◆ Why Desirable

1. **High Cohesion** → Simplifies debugging, improves readability, and reusability.
2. **Low Coupling** → Reduces ripple effects when making changes.
3. **Together** → Improve **maintainability**, **testability**, and **scalability** of the software.

◆ Example

Suppose we have a **Library Management System**:

- A *BookManager* class that only handles book-related logic (high cohesion).
- It interacts with *DatabaseService* through an interface rather than direct access (low coupling).

Thus, if the database technology changes, *BookManager* doesn't need modification.

✓ Mark Distribution

- Definition of cohesion (1)
- Definition of coupling (1)
- Explanation of benefits (2)
- Example (1)

(b) How Dependency Injection (DI) Addresses Tightly Coupled Classes

[5 Marks | CO1]

◆ Problem: Tightly Coupled Classes

When a class directly creates or depends on another class (e.g., `new` keyword), it becomes hard to:

- Replace dependencies (low flexibility)
- Unit test in isolation

- Reuse the class elsewhere

◆ Solution: Dependency Injection

DI **removes the dependency creation** from the class itself and **injects** them externally (e.g., via constructor or framework).

◆ Benefits

1. **Reduces Coupling** – Dependencies are abstracted via interfaces.
2. **Improves Testability** – Can inject mock objects for testing.
3. **Enhances Reusability** – Same class can work with different implementations.
4. **Supports Inversion of Control (IoC)** – The control of object creation is shifted from the class to an external entity or framework.

◆ Example (in Java-like pseudocode)

```
class EmailService {
    void sendEmail() { ... }
}

class UserController {
    private EmailService emailService;

    // Dependency Injection via constructor
    public UserController(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

Here, `UserController` doesn't create `EmailService` — it's injected. Thus, `EmailService` can easily be replaced with a mock for testing.

OR

(a) Model–View–Controller (MVC) Architectural Pattern

[5 Marks | CO1]

Component	Role	Description
Model	Data & business logic	Manages data, rules, and logic of the application.
View	User interface	Displays data from the model and sends user actions to controller.

Component	Role	Description
Controller	Communication bridge	Handles user input, updates model, and refreshes the view.

◆ Interaction Flow

User → View → Controller → Model → View (updated)

◆ Example

In a login system:

- **Model:** Validates user credentials.
- **View:** Displays login form and error messages.
- **Controller:** Handles button clicks, requests validation, and updates view.

(b) SOLID Design Principles

[5 Marks | CO1]

Letter	Principle	Meaning
S	Single Responsibility Principle (SRP)	A class should have one and only one reason to change.
O	Open/Closed Principle (OCP)	Classes should be open for extension but closed for modification.
L	Liskov Substitution Principle (LSP)	Subclasses must be substitutable for their base classes.
I	Interface Segregation Principle (ISP)	Clients should not be forced to depend on methods they don't use.
D	Dependency Inversion Principle (DIP)	High-level modules should depend on abstractions, not on low-level modules.

◆ Example 1 (SRP)

```
class InvoicePrinter {
    void printInvoice(Invoice invoice) { ... } // Only printing
}
```

✓ Single responsibility: printing only.

◆ Example 2 (OCP)

Adding new discount types without modifying existing code using inheritance or interfaces.

🔗 QUESTION 2 [10 Marks]

(a) Objectives of Verification & Validation (V&V)

[5 Marks | CO1]

◆ Objectives

1. Ensure the system meets **specifications (Verification)**.
2. Ensure the final product meets **user needs (Validation)**.
3. Detect defects as early as possible.
4. Improve product quality and reliability.
5. Reduce maintenance and post-deployment cost.

◆ Why Full Validation Cannot Be Achieved by Inspections Alone

- **Inspections** detect static errors in documentation or code (syntax, design issues).
- But **validation** involves **dynamic testing** (runtime behavior, user interaction).
- Many issues (performance, usability, integration) can only be found by executing the software.

✓ Example:

An inspection may verify that all inputs are checked, but only **testing** can validate whether input handling actually prevents crashes.

(b) Three Black-Box Testing Techniques

[5 Marks | CO1]

Technique	Principle	Errors Detected / Example
1. Equivalence Partitioning (EP)	Input data divided into valid and invalid classes; test one from each class.	Detects input validation errors. (<i>e.g., age 1–100 → test 0, 50, 101</i>)
2. Boundary Value Analysis (BVA)	Errors often occur at the boundaries of input ranges.	Detects off-by-one and limit errors. (<i>e.g., for 1–100, test 1, 100, 0, 101</i>)

Technique	Principle	Errors Detected / Example
3. Decision Table Testing	Uses combinations of conditions and actions in tabular form.	Detects logical or rule-based errors (e.g., discount rules).

1. An online banking system suffered a security breach where attackers intercepted [5] communication between users and the server, leading to unauthorized access and data manipulation. Explain how a Man-in-the-Middle (MITM) attack works and suggest preventive technologies and practices that could have mitigated it.

2. A university's student portal allows users to log in using their ID and password. A hacker injects [5] 'OR 1=1 -- into the login field and gains unauthorized access to the admin panel. Explain how this SQL injection attack works and recommend two effective techniques that developers can use to prevent such vulnerabilities in web applications.

5 QUESTION 1 [10 Marks]

(a) Man-in-the-Middle (MITM) Attack

[5 Marks | CO1]

◆ How MITM Attack Works (3 Marks)

A **Man-in-the-Middle (MITM)** attack occurs when a malicious actor secretly intercepts or alters communication between two legitimate parties (e.g., user ↔ bank server) without their knowledge.

◆ Step-by-Step Working

1. **Interception:**
 - Attacker connects to a public Wi-Fi network (e.g., café).
 - When a user connects to the same network, the attacker captures the data packets sent between user and server.
2. **Decryption & Manipulation:**
 - The attacker may **decrypt** unprotected traffic or **impersonate** the server to the client (and vice versa).
 - Sensitive information such as login credentials or transaction details can be **stolen or altered**.
3. **Re-transmission:**
 - The attacker forwards modified or fake responses to the user, appearing as legitimate server data.

◆ Preventive Technologies & Practices (2 Marks)

Preventive Measure	Description
1. Use HTTPS (TLS/SSL Encryption)	Encrypts communication, preventing packet sniffing and tampering.
2. Public Key Infrastructure (PKI)	Ensures authenticity of the server using digital certificates.
3. VPN (Virtual Private Network)	Encrypts all internet traffic even over unsecured Wi-Fi.
4. Avoid Public Wi-Fi for Banking	Use mobile data or secure networks for sensitive transactions.
5. Implement HSTS (HTTP Strict Transport Security)	Forces browsers to only use secure HTTPS connections.

✓ Mark Distribution

- Attack explanation (2)
 - Working steps (1)
 - Preventive technologies (2)
-

(b) SQL Injection Attack in University Portal

[5 Marks | CO1]

◆ How SQL Injection Works (3 Marks)

An **SQL Injection** attack occurs when an attacker inserts **malicious SQL code** into an input field (like login forms), tricking the database into executing unintended commands.

◆ Example Scenario

```
SELECT * FROM users WHERE username = 'admin' AND password = 'OR 1=1 --';
```

Explanation:

- 'OR 1=1 -- makes the condition always true (1=1 is always true).
- The -- symbol comments out the rest of the SQL query.
- As a result, the database returns all users, and the attacker gains unauthorized access, possibly as an **admin**.

◆ Preventive Techniques (2 Marks)

Technique	Description
1. Use Parameterized Queries / Prepared Statements	Prevents user input from being interpreted as code. Example (Java):

```

PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE
username=? AND password=?");
ps.setString(1, username);
ps.setString(2, password);
```


2. Input Validation and Escaping | Validate inputs to allow only expected formats (e.g., alphanumeric). Escape special characters like ` ` or `--`. |

3. Use ORM Frameworks (Optional) | Frameworks like Hibernate auto-sanitize queries. |


```